

SOFTWARE ENGINEERING GROUP
PROJECT

Final Project Presentation

GROUP 05

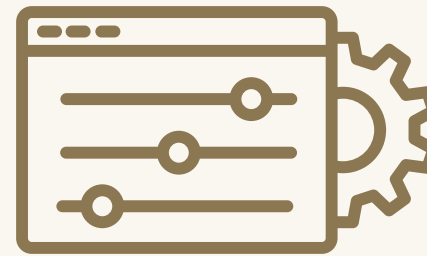
Destiny-Alliah Colombo
Ousman Ndoye
Precious Aghamelu
Laide Olubunmi
Raenah Swaby
Deborah Adekola

The Legal Issue - GDPR Compliance



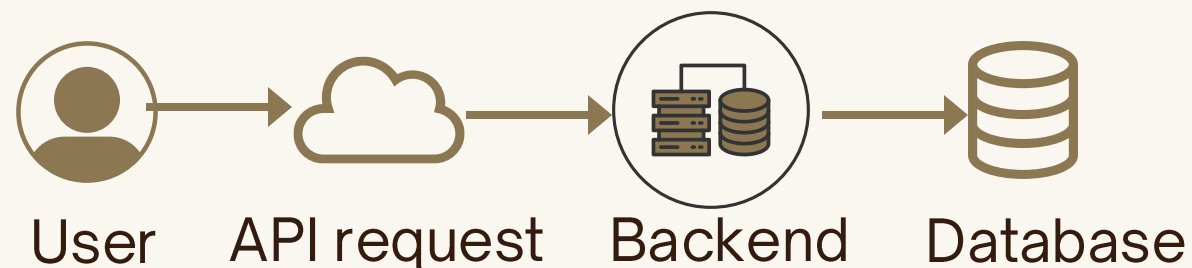
The issue we faced

Our system processed **personally identifiable information** (PII) which is personal data under UK GDPR regulations



Where the issue appears

Personal data is collected during **user registration**, transmitted through API requests and **stored within our backend database**. This means personal data exists across multiple stages of the system lifecycle.



GDPR Obligations and Risks



Lawfulness, fairness & transparency



Data minimisation



Storage limitation



Integrity & Confidentiality



Failure to comply would result in a personal data breach for users of our system

Why It Matters

Legal Liability & Financial Risk



Improper handling of **personal data** can result in a personal **data breach** under UK GDPR.
This exposes the organisation to **fines and legal liability**.

Security & System Integrity



Authentication mechanisms such as OAuth2 **tokens** are critical **access points** within the system.
If stolen, they allow attackers to impersonate users through **session hijacking**.

User Trust & System Adoption



Users must **trust** that their personal data is handled **securely and responsibly**.
Any breach of data can lead to **loss of user confidence** and long-term **reputational damage** to the system.

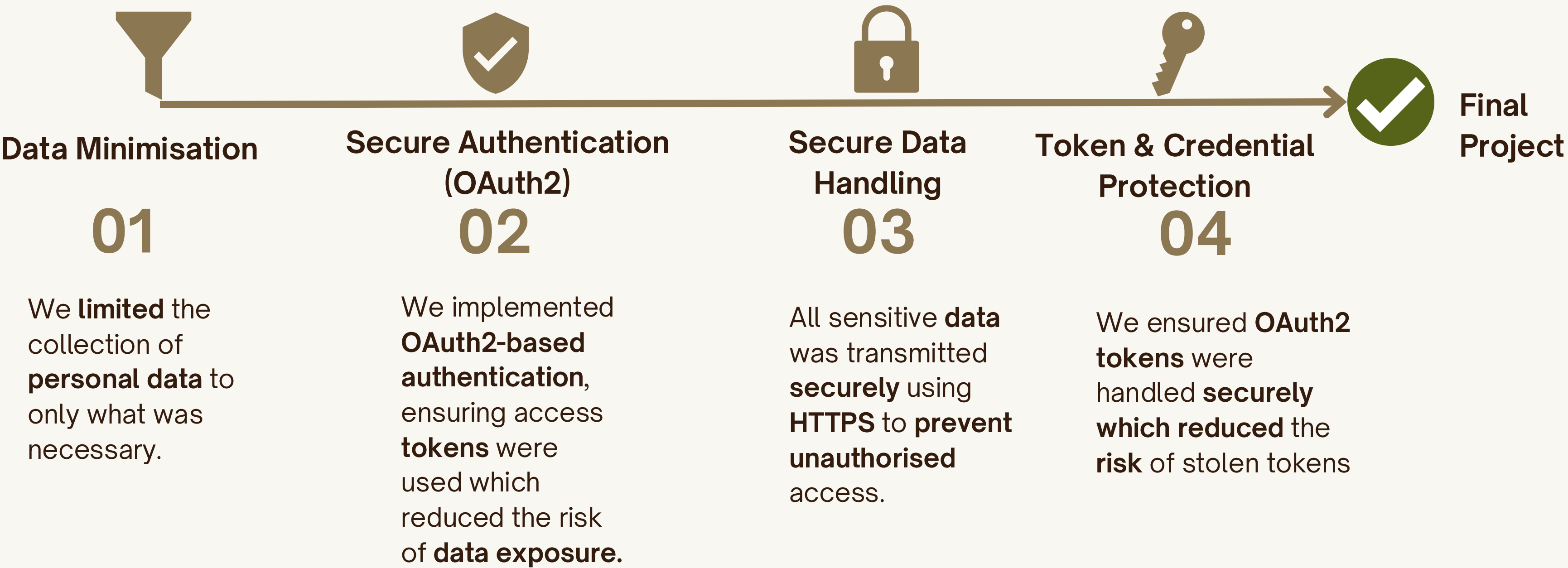


Key Insight

GDPR compliance is not separate from system design — it directly defines how secure and trustworthy the system is.

What We Did - Ensuring GDPR Compliance

DESTINY-ALLIAH
COLOMBO



We integrated GDPR requirements into our system architecture and security decisions

How we could improve

DESTINY-ALLIAH
COLOMBO



1. User Right to Erasure

Under UK GDPR, users have the “right to be forgotten”, which means that they should be able to request to delete of their personal data.

Our system currently does not provide a direct feature for users to delete their account and personal data.

This feature would allow users to permanently remove their data, improving GDPR compliance and giving users more control on their data.

Our Goal



Our goal is to build a system that not only protects data, but also empowers users and fully aligns with GDPR principles.



Supporting Improvements



2. Data Retention Policies

Introduce automatic data retention policies to ensure that personal data is not stored longer than necessary.



3. Token Expiration

An improvement could be stricter OAuth2 token expiry regulations, reducing the risk of token misuse.

What is the purpose of this project?

Expectations

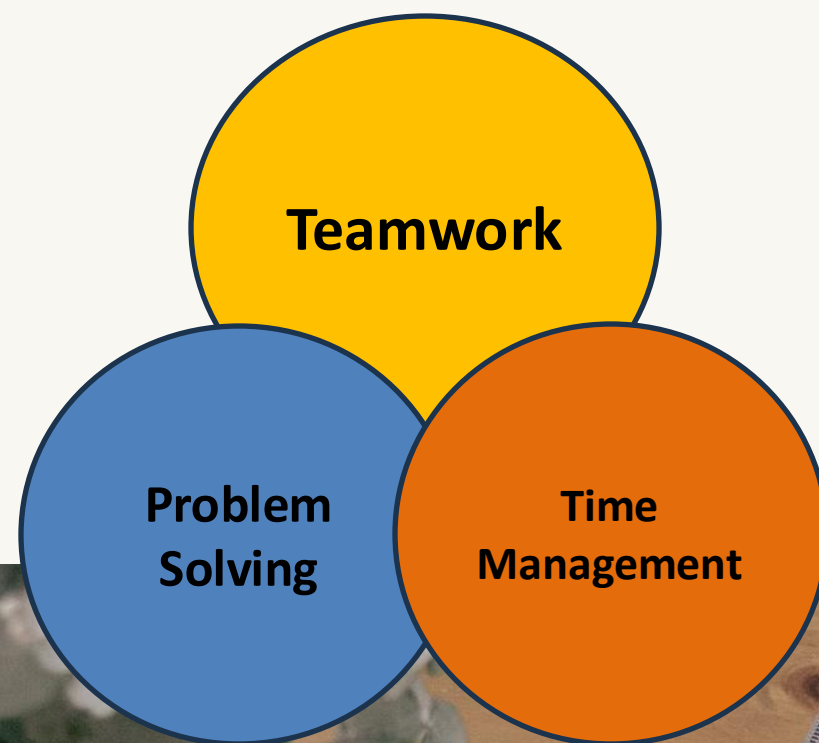


- Stay focused
- Achieve tasks on time
- Unified goals
- Shared understanding and competency



Reality

- Distracted and affected by external factors
- Struggle with managing time and deadlines
- Different opinions on approach and results
- Gaps in knowledge, needs help but doesn't ask for it



Accountability



Objectives and
Results

Team
Feedback

Reflection
& Growth

Achieved
Goals

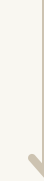


The initial objective was to develop a gamified companion to IBM's Skills build courses

The Result was a project we were all proud of, Including meaningful commits, clear user manuals and achieved User Stories verified by clear Definition of Dones



- Feedback brought grey areas to light and allowed us to overcome obstacles as a team
- Any issue or error was a shared responsibility, those who didn't understand could learn and other members lead with example and developed trust



Realisation that the project wasn't the only thing developed. Soft Skills were strengthened and diverse perspectives were combined which allowed us to share knowledge

Constructive Feedback

Not just Anything

Feedback should be bidirectional, not only offered but also received openly. They should reference shared goals and focus on improvement, not just shift blame

What It Does

- Improves performance
- Increased personal development
- Stronger relationships
- Boosts motivation and drive
- Error handling and prevention



Software Design choices and why we chose them:

The technologies used:

The Programming languages used:



Java + Spring boot
Backend Framework



JSP / Thymeleaf
Server-side templating



HTML & CSS
Frontend structing & styling

Platforms and tools used:



IntelliJ IDEA
Java IDE with spring framework



Gitlab
Version control & scrum board



Microsoft Teams
Communications + Meetings

Java + Spring



JSP Templates



HTML/CSS (Browser)

Software Design choices and why we chose them:

Why use Java + Spring boot ?



Spring setup and starters:

Built-in security, OAuth2, JPAs, password encoding etc. This allowed us to on features not infrastructure



Java static feature (debugging):

Compiler catches bugs before running time, which was extremely critical with 6 contributors across services, model, and branches



Intelli j + Spring + Java integration:

Java being native allowed for native IDE integration, such as; refactoring, debugging and auto-completion that other programming languages lacked in this ecosystem

Vs JavaScript, Python or C++?

- Java offers more comfortability over **C++**.
- **Python** was not suited for the logically complexities of user stories.
- **JavaScript (JS)** does offer more flexibility on the client-side server. However, for our server-side architecture Java offered us more:
 - Mature ecosystem
 - Team comfortability
 - Intelli j + GitLab linking
 - Gamified features creation ease

Using predominantly Java allowed us to provide the backend robustness required for both sprints.

Software Design choices and why we chose them:

Frontend + Client-side rendering and Platforms justifications:



Justifications for Platform choices



What allowed for Team collaboration?

- ❖ **Intelli J** allowed for built-in spring support, database integration, while also offering "Code with me" sessions
- ❖ **Gitlab** allowed for ease of coding and debugging across everyone's platform due to separate but linked branches, merge request, resolving conflicts etc.
- ❖ Meetings on **Microsoft Teams** strengthened our team collaboration, as scheduled meetings was used to track progress.



Why didn't other popular platforms work?

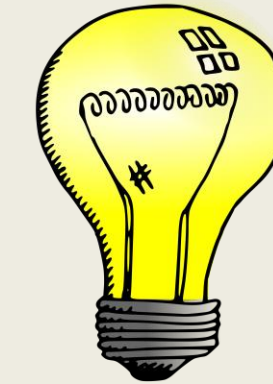
- ❖ While with **VS code** (as a coding platform), the spring support and database integration – although achievable – would require more steps.
- ❖ **VS Code** offers GitLab access but lacks the native Spring integration IntelliJ provides.
- ❖ Teams was just an easier choice as we already owned Teams accounts and apps like **Zoom**, would be rather cumbersome.

Software Design choices and why we chose them:

Reflection:

What would I reconsider and do differently?

- ✓ Employ more JavaScript for async interactions and more UI animations. As I recognise how important that would have been in a gamified webapp.
- ✓ Exploring reusable component patterns earlier so we weren't repeating similar layouts across pages, as that was a problem I ran into far too late in my code development.
- ✓ Finally, Spend more time getting familiar with what IntelliJ had to offer to allow for ease of code.



Any Key Takeaway?

- ❖ Learnt more about the industry standards and relevance of web development, especially when it came to the usage of:
 - Gitlab + Scrum
 - JPA Repositories
 - Usage of IntelliJ and Spring frameworks
 - Microsoft teams for hosting and scheduling meetings

Why communication matters in a group project:

1. Clear division of tasks

- Communication helps teams understand each member's skills, experience, and strengths
- This allows tasks to be delegated to those best suited to complete them, improving efficiency
- Balanced task delegation reduces frustration and improves the overall quality of the final project



2. Improves problem solving

- Programming challenges such as bugs and merge issues are inevitable in group projects
- Communication allows members to ask questions, share solutions, and draw on each other's experience
- This openness prevents problems from worsening and improves both productivity and technical understanding across the team

3. Deadlines and project quality

- Regular communication ensures the team can track progress, identify delays early, and avoid last-minute rushes before deadlines
- Without consistent updates, incomplete tasks can go unnoticed, leading to panic, rushed code, and unfinished features
- Team discussions also enable code reviews, helping to catch mistakes, improve readability, and maintain consistent style across the Java project, increasing the likelihood of submitting a polished and functional application on time



Our Communication Methods:

1. Group chat

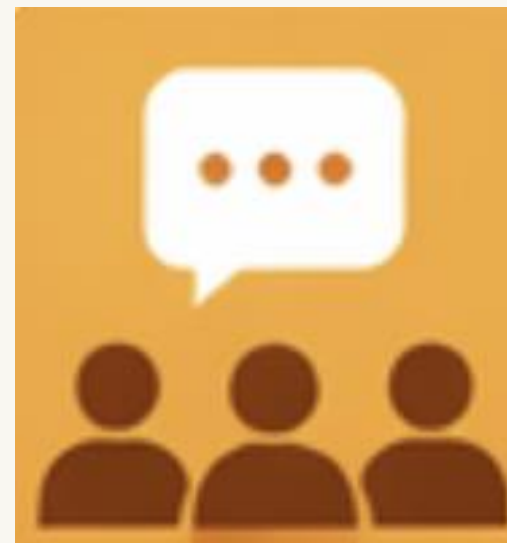
Weekly



- An iMessage group chat served as our primary communication tool, used to share progress updates and keep all members informed throughout the project
- The chat allowed members to ask questions about each other's code, coordinate meeting times
- Any issues or bugs were shared promptly in the chat, allowing the team to troubleshoot together

2. In person meetings

Monthly



- In-person meetings were held to plan sprint structure and align the team on priorities and build a shared understanding of each component
- Supervisor feedback was reviewed and discussed as a group
- These meetings encouraged open discussion, helping the team identify issues early and make informed decisions throughout the project

3. Checkpoint Labs

Fortnightly



- Checkpoint labs enabled practical progress, including establishing a working development environment
- Progress was demonstrated to the supervisor, who provided direct feedback
- These sessions kept the team accountable, ensured key technical foundations were in place, and allowed the supervisor to verify progress at each stage

What Went Well:

1. Honesty

- Team members communicated openly when tasks were completed, allowing the next stage of work to begin.
- Members were comfortable admitting when they needed support, enabling quicker problem solving.
- Honest communication about availability allowed the team to adjust expectations realistically, with planning built on trust rather than assumptions about capacity.

2. Member Support

- When problems arose, the team discussed them openly across the group chat ensuring no member was left to handle issues alone
- Members gave clear updates on who was supporting which tasks, ensuring collaboration was organised and targeted effectively
- Errors were communicated and explained promptly rather than waiting for meetings, allowing the team to identify causes, test solutions, and prevent repeated mistakes efficiently

3. Demonstrations

- Members demonstrated working code outputs rather than simply reporting tasks as complete, keeping the team confident in the direction of development
- Demonstrations were linked back to project criteria, ensuring features directly satisfied assessment goals
- Sharing videos in the group chat allowed progress to be reviewed and created opportunities for constructive feedback that improved usability and quality

What Went Wrong and What We Learnt:

1. Inconsistency

- Communication was quite consistent but became most active only when deadlines approached, making it reactive rather than a tool for early planning.
- The group learnt that timing of communication matters more than frequency where the use of timely conversations prevent missing internal deadlines.
- Next time, the team would introduce mid-point reviews tied to specific web app milestones.

2. Limited In-person meetings

- Infrequent in-person meetings meant the team was limited in genuine collaboration, with the project feeling more like individual work being combined rather than a shared effort.
- Without regular face-to-face interaction, opportunities to generate ideas together, divide tasks clearly, and build team cohesion were reduced.
- More frequent in-person sessions would have fostered a stronger sense of collaboration

3. Task assignment

- Tasks were assigned loosely, meaning some members adopted more work than others.
- This created uneven workload distribution meaning the project succeeded but individual development was uneven.
- A shared visual progress tracker reviewed during team check-ins would make contributions transparent.

OAuth2 Social Login

Industry-standard framework for delegated authentication



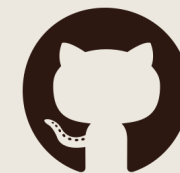
Continue with GitHub

Username

Password



User

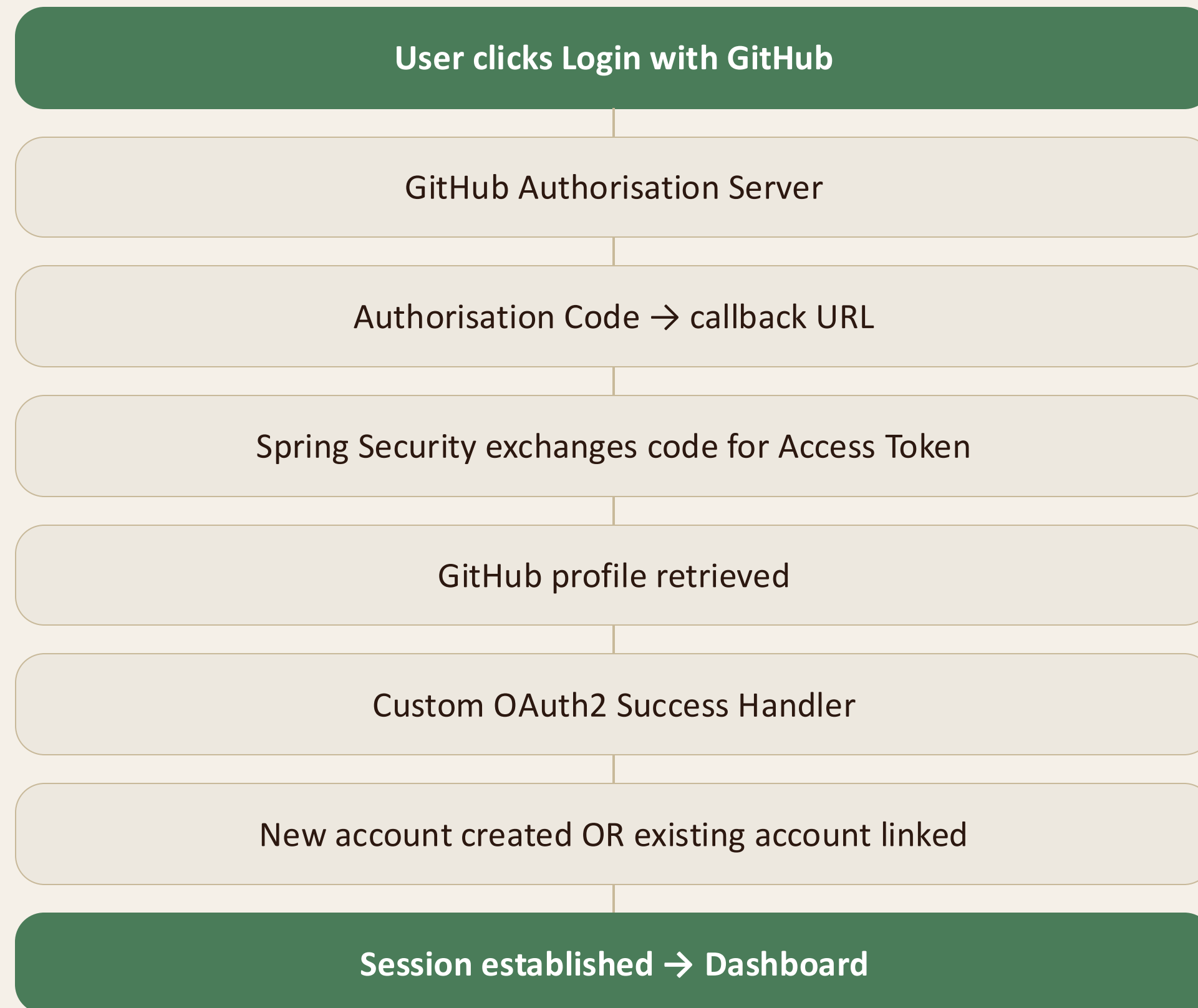


GitHub



Your App

Authentication Flow



The Identity Resolution Bug

The Problem

auth.getName() returns different values for form login vs OAuth2 users

Form login → returns username (matches DB)

OAuth2 login → returns GitHub principal ID (no match)

The Fix

OAuth2-aware helper method checks principal type before lookup

If OAuth2User → look up by GitHub provider ID

If form login → look up by username as before

```
if (principal instanceof OAuth2User oauthUser) {  
    String providerId = oauthUser.getAttributes().get("id");  
    return userRepo.findByProviderAndProviderId("GITHUB", providerId);  
}  
  
return userRepo.findByUsername(authentication.getName());
```



Key Takeaways

01

Third-party auth requires consistent identity resolution across the whole application

02

Inconsistent user lookup is not just a bug — it is a potential security vulnerability

03

Centralise authentication logic from the start to prevent controller-level inconsistencies

TECHNICALLY CHALLENGING FEATURE: GAMIFICATION SYSTEM DESIGN

Deborah Adekola

THE PROBLEM

Maintaining User Engagement

- Users often lose motivation over time in static learning systems
- Lack of feedback reduces interaction and consistency
- Traditional platforms do not actively encourage progress
- Needed a system to motivate users and reward activity

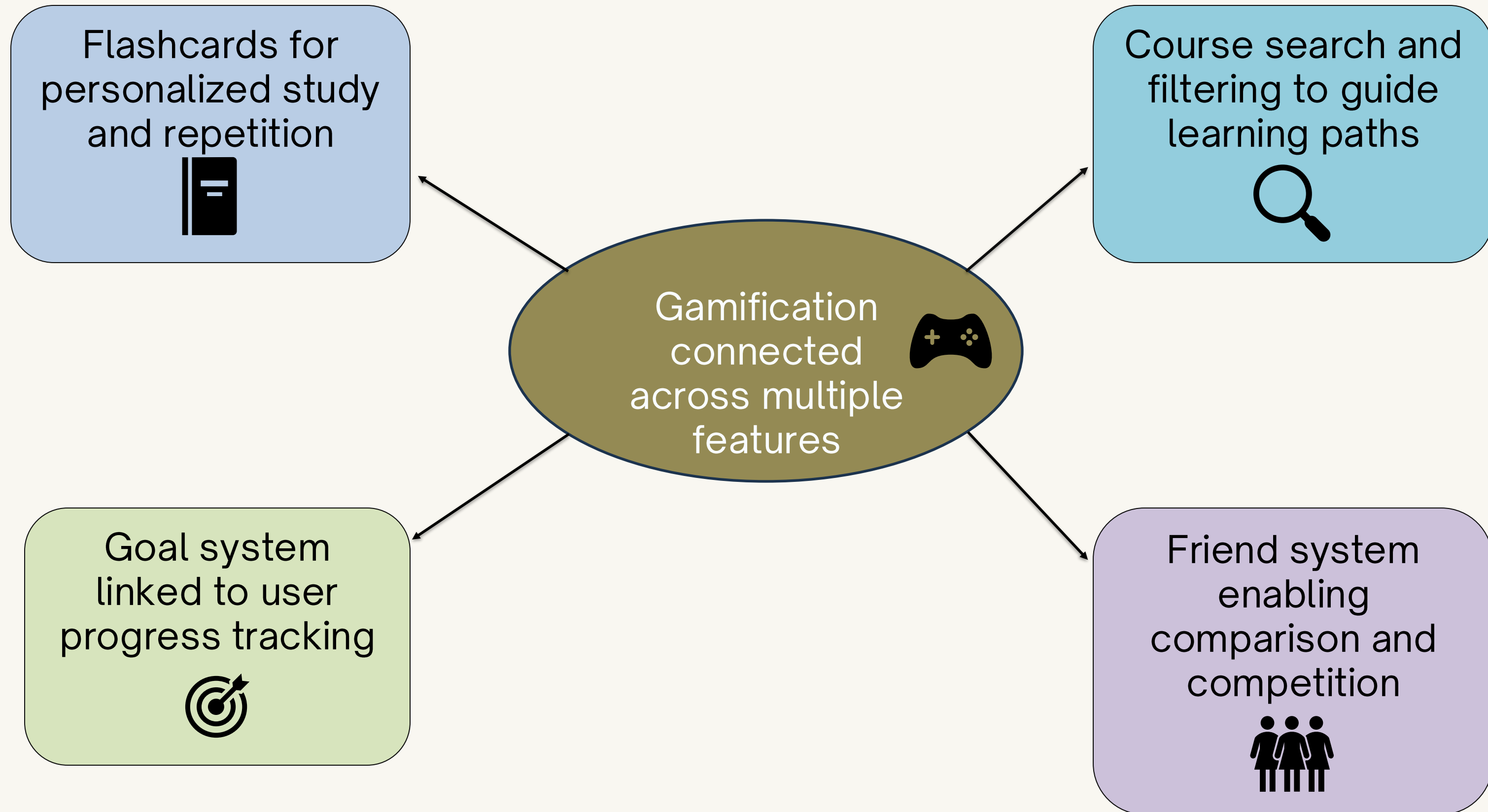
OUR SOLUTION

Integrated Gamification System

- Implemented progress tracking across user activities
- Introduced goal-setting with visible progress on dashboard
- Developed leaderboards (global and friends) to encourage completion
- Provided feedback through rewards and completion indications

SYSTEM INTEGRATION

Deborah Adekola



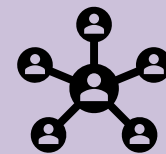
WHY IT WAS CHALLENGING

Deborah Adekola



SYSTEM COMPLEXITY

- Multiple features interacting
- Backend + frontend coordination
- Managing system-wide logic.



INTEGRATION

- Features depended on each other
- Data shared across components
- Needed consistent updates

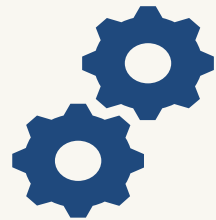


DESIGN BALANCE

- Simplicity vs Functionality
- Avoiding overcomplication
- Keeping the user experience clear

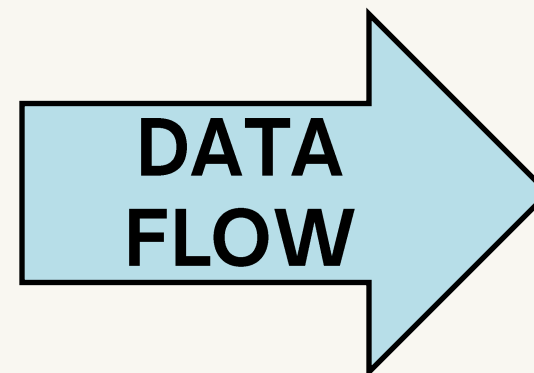
TECHNICAL IMPLEMENTATION

Deborah Adekola



BACKEND (Spring boot)

- Stores user progress
- Handles goals & rewards
- Updates leaderboard data
- Manages system logic



FRONTEND (UI)

- Displays dashboard
- Shows leaderboards
- Visual feedback (progress)
- Dark/Light mode support

We made sure to design the system to be scalable and adaptive

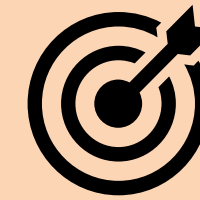
KEY CHALLENGES

Deborah Adekola



CHALLENGE

- Integrating multiple features without breaking functionality
- Managing different team ideas and design approaches
- Avoiding overly complex or cluttered user experience
- Ensuring data consistency across backend and frontend



HOW WE ADDRESSED IT

- Broke features into smaller parts and tested each feature incrementally
- Discussed and prioritized best solutions and focused on project goals
- Simplified logic and UI design, we focused on core features

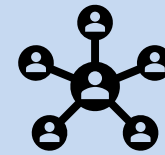
WHAT WE LEARNED

Deborah Adekola



MEANINGFUL DESIGN

- Gamification must have a clear purpose
- Not just visual but behavior-driven



SYSTEM INTEGRATION

- Features must work together, not separately
- Backend + frontend must stay consistent



SIMPLICITY MATTERS

- Simple systems are more effective
 - Avoid overengineering. Focus on usability

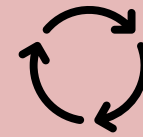
WHY IT MATTERS

Deborah Adekola



USER ENGAGEMENT

- Encourages regular interaction
- Provides motivation through rewards



RETENTION

- Keeps users coming back consistently
- Builds long-term usage habits



REAL-WORLD DESIGN

- Reflects modern apps (e.g. Duolingo)
- Demonstrates full system integration

Studio ShodwE

The End

GROUP 05

Destiny-Alliah Colombo
Ousman Ndoye
Precious Aghamelu
Laide
Raenah Swaby
Deborah Adekola